



Node.js & Asynchronous JavaScript

INSTRUCTOR NAME: SANTHANALAKSHMI

INSTITUTION NAME: CHRIST UNIVERSITY

CLASS : 5BTCSIOT

DATE OF SUBMISSION: 27/08/2026

Practical Question Paper - Set 2

<u>NAME</u>	<u>REGISTRATION NUMBER</u>	<u>CHRIST EMAIL ID</u>
BARATH C G	2462507	barath.cg@btech.christuniversity.in

Department of Computer Science and Engineering

School of Engineering and Technology

CHRIST (Deemed to be University)

Kumbalgodu, Bangalore - 560 074

August-2026

27/8/26

Node.js & Asynchronous Javascript

BARATH C G
2462507
SBTCS1PT

Theory Question Paper - Set B

- Q1). Node.js will show an error like `Error: Cannot find module '.../app.js'`. The developer should move to correct folder containing `app.js` or provide correct file path.
- Q2). V8 executes Javascript code, libuv handles asynchronous operations such as network requests, event loop & file operations. Together, they allow Node.js to execute JavaScript while handling I/O asynchronously.
- Q3). The fs documentation provides method syntax, parameters, return values, and examples. It helps developers choose between synchronous methods like `readFileSync()` and asynchronous methods like `readFile()`.
- Q4).

```
[  
  'path/to/node',  
  'path/to/app.js',  
  'hello',  
  'world'  
]
```

 The first two values are Node.js executable - path & script path followed by command line argument.
- Q5). `getUser()` returns a Promise, so `user` is not the actual user object. Use `await` to get resolved value.
- ```
const user = await getUser();
console.log(user.name);
```
- Q6). 

```
const fs = require('fs');

function readFileCallback(path, callback) {
 fs.readFile(path, 'utf8', (err, data) => {
 if (err) return callback(err, null);
```

```

 callback(null, data);
 }
}

```

Q10). Function function delay(ms) {  
 return new Promise((resolve) => {  
 setTimeout(resolve, ms);  
 });  
}

Q11). Async function f() {  
 try {  
 await Promise.reject('err1');  
 } catch (e) {  
 console.log('Caught: ', e);  
 }  
}

Answer: Caught: err1

Q12). function a() {  
 console.log('a');  
 b();  
}

function b() {  
 console.log('b');  
 a();  
 console.log('c');  
}

Answer: a  
 b  
 c

Q15). math.js

```

function add(a, b) {
 return a + b;
}
module.exports = add;

```

app.js

```

const add = require('./math');
console.log(add(2, 3));

```

# Countdown Timer & Notification App

*Node.js & Asynchronous JavaScript CIA-2 Practical Answers - 10 Tasks*

|               |                                          |
|---------------|------------------------------------------|
| Topic         | Countdown Timer & Notification App       |
| Tasks Covered | Tasks 1, 2, 3, 4, 5, 8, 9, 10, 11 and 13 |
| Language Used | JavaScript with Node.js                  |

## Task 1: Countdown App Setup and First Run

**Question:** Initialize a new Node.js project using `npm init -y` and create an entry file `countdown.js`. Run `countdown.js` using `node countdown.js` and confirm it prints "Countdown App Ready".

### Code Screenshot



```
task01_countdown.js JavaScript
1 // countdown.js
2 // Project setup command: npm init -y
3
4 console.log("Countdown App Ready");
```

### Output Screenshot



```
Terminal - Node.js PowerShell
PS> npm init -y
Wrote to countdown-app\package.json
PS> node countdown.js
Countdown App Ready
```

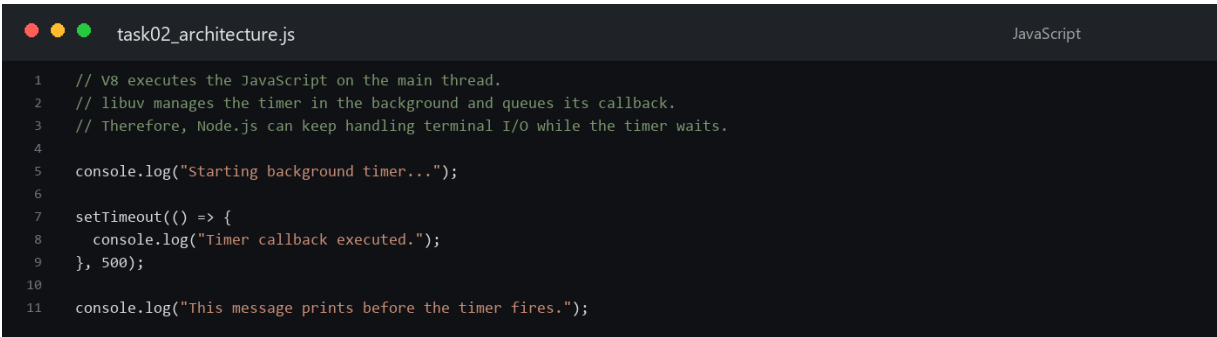
**Summary:** Initialized the project workflow and created the countdown entry file. The successful run confirms that Node.js executes the file and prints the required readiness message.

**Used:** `npm init -y`, Node.js command line, `console.log()`

## Task 2: Node.js Architecture: V8 and libuv

**Question:** Add a comment explaining how V8 and libuv let countdown.js keep accepting terminal input while a timer runs in the background. Demonstrate this by starting a `setTimeout` and immediately printing another message before the timer fires.

### Code Screenshot



```
task02_architecture.js JavaScript
1 // V8 executes the JavaScript on the main thread.
2 // libuv manages the timer in the background and queues its callback.
3 // Therefore, Node.js can keep handling terminal I/O while the timer waits.
4
5 console.log("Starting background timer...");
6
7 setTimeout(() => {
8 console.log("Timer callback executed.");
9 }, 500);
10
11 console.log("This message prints before the timer fires.");
```

### Output Screenshot



```
Terminal - Node.js PowerShell
PS> node task02_architecture.js
Starting background timer...
This message prints before the timer fires.
Timer callback executed.
```

**Summary:** V8 executes the JavaScript while libuv manages the scheduled timer and later queues its callback. The immediate message appears before the timer callback, demonstrating non-blocking timer behavior.

**Used:** V8, libuv, `setTimeout()`, event loop



## Task 3: Timers Documentation and Basic Countdown

**Question:** Refer to the official Node.js documentation for the timers module and list, as a comment, the method names used. Adapt an example to build a basic countdown using `setInterval`.

### Code Screenshot

```
task03_basic_countdown.js JavaScript
1 // Official Node.js timers methods used:
2 // setInterval() and clearInterval()
3
4 let secondsRemaining = 3;
5 console.log(`Countdown starts at ${secondsRemaining} seconds`);
6
7 const intervalId = setInterval(() => {
8 secondsRemaining -= 1;
9 console.log(`${secondsRemaining} second(s) remaining`);
10
11 if (secondsRemaining === 0) {
12 clearInterval(intervalId);
13 console.log("Countdown complete!");
14 }
15 }, 1000);
```

### Output Screenshot

```
Terminal - Node.js PowerShell
PS> node task03_basic_countdown.js
Countdown starts at 3 seconds
2 second(s) remaining
1 second(s) remaining
0 second(s) remaining
Countdown complete!
```

**Summary:** Used `setInterval()` to update the remaining time every second and `clearInterval()` to stop exactly at zero. The method names are documented in comments as required.

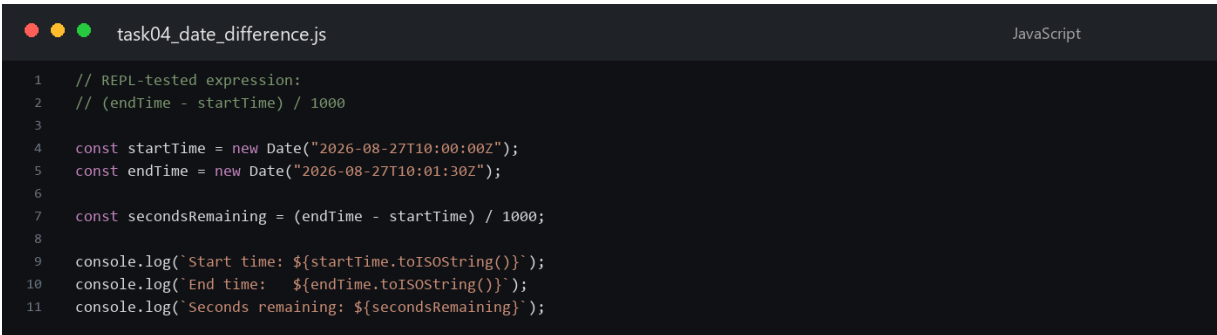
**Used:** `setInterval()`, `clearInterval()`

**Official reference:** [Node.js Timers documentation](https://nodejs.org/docs/latest/api/timers)

## Task 4: Node REPL and Date Difference

**Question:** In the Node REPL, test a snippet that calculates the seconds remaining between two Date objects. Move the working snippet into countdown.js.

### Code Screenshot



```
task04_date_difference.js JavaScript
1 // REPL-tested expression:
2 // (endTime - startTime) / 1000
3
4 const startTime = new Date("2026-08-27T10:00:00Z");
5 const endTime = new Date("2026-08-27T10:01:30Z");
6
7 const secondsRemaining = (endTime - startTime) / 1000;
8
9 console.log(`Start time: ${startTime.toISOString()}`);
10 console.log(`End time:   ${endTime.toISOString()}`);
11 console.log(`Seconds remaining: ${secondsRemaining}`);
```

### Output Screenshot



```
Terminal - Node.js PowerShell
PS> node task04_date_difference.js
Start time: 2026-08-27T10:00:00.000Z
End time: 2026-08-27T10:01:30.000Z
Seconds remaining: 90
```

**Summary:** Subtracted two Date objects and divided the millisecond difference by 1000. The same REPL-tested expression reports 90 seconds when executed from the file.

**Used:** Node REPL, Date objects, arithmetic conversion

## Task 5: Command-Line Duration and Cancel Input

**Question:** Modify `countdown.js` to accept the countdown duration in seconds through `process.argv`. Use `process.stdin` so the user can type "cancel" to stop the countdown early.

### Code Screenshot



```
1 const duration = Number(process.argv[2]);
2
3 if (!Number.isFinite(duration) || duration <= 0) {
4 console.error("Enter a positive duration in seconds.");
5 process.exitCode = 1;
6 } else {
7 let remaining = duration;
8 console.log(`Countdown started for ${duration} seconds.`);
9 console.log('Type "cancel" and press Enter to stop it.');
```

### Output Screenshot



```
Terminal - Node.js
PS> "cancel" | node task05_cli_cancel.js 5
Countdown started for 5 seconds.
Type "cancel" and press Enter to stop it.
Countdown cancelled early.
```

**Summary:** Read the duration from `process.argv` and listened for terminal data through `process.stdin`. Entering `cancel` clears the interval and stops the countdown before zero.

**Used:** `process.argv`, `process.stdin`, `clearInterval()`



## Task 8: Callback-Based Asynchronous Function

**Question:** Write `checkTimeLeftCallback(seconds, callback)`, simulate a delay with `setTimeout`, then call it and log the remaining time inside the callback.

### Code Screenshot



```
task08_callback.js JavaScript
1 function checkTimeLeftCallback(seconds, callback) {
2 // Simulate an asynchronous operation before returning the value.
3 setTimeout(() => {
4 if (seconds < 0) {
5 callback(new Error("Seconds cannot be negative"), null);
6 return;
7 }
8 }, 300);
9 callback(null, seconds);
10 }
11
12
13 checkTimeLeftCallback(5, (error, timeLeft) => {
14 if (error) {
15 console.error(error.message);
16 return;
17 }
18 console.log(`Time left: ${timeLeft} seconds`);
19 });
20
```

### Output Screenshot



```
Terminal - Node.js PowerShell
PS> node task08_callback.js
Time left: 5 seconds
```

**Summary:** Created an error-first callback function that waits asynchronously before returning the remaining seconds. The callback handles either an error or the successful time value.

**Used:** callback function, `setTimeout()`, error-first callback pattern

## Task 9: Countdown with Node Timers

**Question:** Use `setInterval` to print the remaining seconds every second and stop it with `clearInterval` exactly at zero. Use a separate `setTimeout` to trigger a "Time's up!" notification after the countdown ends.

### Code Screenshot



```
task09_timers.js JavaScript
1 let remaining = 3;
2 console.log(`Remaining: ${remaining} seconds`);
3
4 const intervalId = setInterval(() => {
5 remaining -= 1;
6 console.log(`Remaining: ${remaining} seconds`);
7
8 if (remaining === 0) {
9 clearInterval(intervalId);
10 console.log("Interval stopped at zero.");
11 }
12 }, 1000);
13
14 setTimeout(() => {
15 console.log("Time's up!");
16 }, remaining * 1000 + 50);
```

### Output Screenshot



```
Terminal - Node.js PowerShell
PS> node task09_timers.js
Remaining: 3 seconds
Remaining: 2 seconds
Remaining: 1 seconds
Remaining: 0 seconds
Interval stopped at zero.
Time's up!
```

**Summary:** The interval prints each remaining value and is cleared at zero. A separate timeout displays the final notification immediately after the countdown finishes.

**Used:** `setInterval()`, `clearInterval()`, `setTimeout()`

## Task 10: Promise-Based Time Check

**Question:** Rewrite `checkTimeLeftCallback` as `checkTimeLeftPromise(seconds)`. Chain `.then()` and `.catch()` to log the result or an error.

### Code Screenshot

```
task10_promise.js JavaScript
1 function checkTimeLeftPromise(seconds) {
2 return new Promise((resolve, reject) => {
3 setTimeout(() => {
4 if (!Number.isFinite(seconds) || seconds < 0) {
5 reject(new Error("Invalid countdown duration"));
6 return;
7 }
8 }, 3000);
9 resolve(seconds);
10 });
11 }
12
13 checkTimeLeftPromise(8)
14 .then((timeLeft) => {
15 console.log(`Promise resolved: ${timeLeft} seconds remaining`);
16 })
17 .catch((error) => {
18 console.error(`Promise rejected: ${error.message}`);
19 });
20 }
```

### Output Screenshot

```
Terminal - Node.js PowerShell
PS> node task10_promise.js
Promise resolved: 8 seconds remaining
```

**Summary:** Wrapped the delayed check in a Promise that resolves valid durations and rejects invalid values. The result is handled using a `.then()/catch()` chain.

**Used:** Promise, `resolve()`, `reject()`, `.then()`, `.catch()`

## Task 11: Async-Await with Try/Catch

**Question:** Write `runCountdownAsync(seconds)`, await `checkTimeLeftPromise(seconds)` inside `try/catch`, and demonstrate the catch block by passing a negative duration.

### Code Screenshot

A screenshot of a code editor window titled 'task11\_async\_await.js' with a 'JavaScript' language identifier. The code defines a function `checkTimeLeftPromise` that returns a Promise. Inside the Promise, a `setTimeout` is used to reject the Promise with an error 'Duration must be zero or greater' if the duration is not finite or negative. Then, an async function `runCountdownAsync` is defined, which uses `try/catch` to await `checkTimeLeftPromise` and log the result or error. Finally, `runCountdownAsync(-2)` is called to demonstrate the error handling.

```
1 function checkTimeLeftPromise(seconds) {
2 return new Promise((resolve, reject) => {
3 setTimeout(() => {
4 if (!Number.isFinite(seconds) || seconds < 0) {
5 reject(new Error("Duration must be zero or greater"));
6 return;
7 }
8 resolve(seconds);
9 }, 300);
10 });
11 }
12
13 async function runCountdownAsync(seconds) {
14 try {
15 const timeLeft = await checkTimeLeftPromise(seconds);
16 console.log(`Async result: ${timeLeft} seconds remaining`);
17 } catch (error) {
18 console.log(`Caught error: ${error.message}`);
19 }
20 }
21
22 runCountdownAsync(-2);
```

### Output Screenshot

A screenshot of a terminal window titled 'Terminal - Node.js' with a 'PowerShell' language identifier. It shows the command `PS> node task11_async_await.js` being executed, followed by the output `Caught error: Duration must be zero or greater`.

```
Terminal - Node.js
PowerShell

PS> node task11_async_await.js
Caught error: Duration must be zero or greater
```

**Summary:** Awaited the Promise inside a protected try block. Passing `-2` rejects the Promise, so the catch block prints the validation error.

**Used:** `async`, `await`, `try/catch`, Promise rejection

## Task 13: Event Loop and Job Queue Order

**Question:** Combine `setTimeout`, a resolved Promise `.then()`, and a synchronous `console.log`. Predict the output order in a comment, run it, and confirm whether the prediction was correct.

### Code Screenshot

```
task13_event_loop.js JavaScript
1 // Predicted order:
2 // 1. Synchronous message
3 // 2. Promise microtask
4 // 3. setTimeout callback
5
6 setTimeout(() => {
7 console.log("3. setTimeout callback");
8 }, 0);
9
10 Promise.resolve().then(() => {
11 console.log("2. Promise microtask");
12 });
13
14 console.log("1. Synchronous message");
15
16 // The observed output confirms the prediction because the Promise job
17 // queue is processed before the timer callback queue.
```

### Output Screenshot

```
Terminal - Node.js PowerShell
PS> node task13_event_loop.js
1. Synchronous message
2. Promise microtask
3. setTimeout callback
```

**Summary:** The synchronous log runs first, followed by the Promise microtask and then the timer callback. The observed output matches the prediction written in the code comments.

**Used:** call stack, Promise job queue, timer callback queue, event loop

**GITHUB link:** <https://github.com/BarathCG/L-T-SEM-5>